

计算机体系结构

新型计算机研究所

陈 衡

西一楼 A 段415室

Email: hengchen@xjtu.edu.cn

QQ群号: 1071090574

第1章 计算机系统结构的基础知识

- ◆ 1.1 计算机系统结构的基本概念
- ◆ 1.2 计算机系统的设计
- ◆ 1.3 计算机系统的性能评测
- ◆ 1.4 计算机系统结构的发展

1.1 计算机系统结构的基本概念

- ◆ 第一台通用电子计算机诞生于1946年
- ◆ 计算机技术的飞速发展得益于两个方面
 - 计算机制造技术的发展
 - 计算机系统结构的创新
- ◆ 经历了4个发展过程



1.1 计算机系统结构的基本概念

时间	原因	每年的性能增长
1946年起的25年	两种因素都起着主要的作用	25%
20世纪70年代末—80年代初	大规模集成电路和微处理器出现，以集成电路为代表的制造技术的发展	约35%
20世纪80年代中开始	RISC结构的出现，系统结构不断更新和变革，制造技术不断发展	50%以上维持了约16年
2003年以来	功耗问题；可以进一步有效开发的指令级并行性已经很少；存储器访问速度的提高缓慢	约20%

1.1 计算机系统结构的基本概念

- ◆ 系统结构的重大转折：
 - 从单纯依靠**指令级并行**转向开发**线程级并行**和**数据级并行**。
- ◆ 为什么要学习计算机体系结构？
 - 开展体系结构研究与设计的基础
 - 更好地设计与实现操作系统、编译器
 - 更好地设计与实现应用程序
 - 评测计算机系统的性能
 - 理解时间、空间、价格成本之间的权衡关系

1.1.1 计算机系统的层次结构

- ◆ 计算机系统的层次结构
 - 计算机系统=硬件/固件+软件
- ◆ 计算机语言从低级向高级发展
 - 高一级语言的语句相对于低一级语言来说功能更强，更便于应用，但又都以低级语言为基础。
- ◆ 从计算机语言的角度，把计算机系统按功能划分成多级层次结构。
 - 每一层以一种语言为特征

1.1.2 计算机系统结构的定义

◆ 什么是计算机系统结构？

- 描述计算机系统的功能、组织和实现的一组规则和方法。
- 计算机体系结构是研究如何选择（设计）功能部件和互联方法来满足计算机系统的功能、性能、价格约束的科学
- 计算机体系结构是软件设计者与硬件设备(VLSI)之间的中间层，是软件与硬件的接口 (Interface)

1.1.2 计算机系统结构的定义

过去的定义：

传统机器程序员可见的计算机系统的属性。包括：**概念性的结构**和**功能特性**。不包括：数据流和控制流的组织、逻辑设计以及物理实现。

– Amdahl, Blaauw, and Brooks, 1964

Instruction-Set Architecture

广义的定义：

计算机体系结构研究如何 **设计、组织，以及使用可用的生产技术实现一个计算系统**，该计算系统可有效地执行软件应用，满足价格、功耗和性能约束

包括：指令系统结构、组成、硬件

1.1.2 计算机系统结构的定义

◆ 过去的观点：

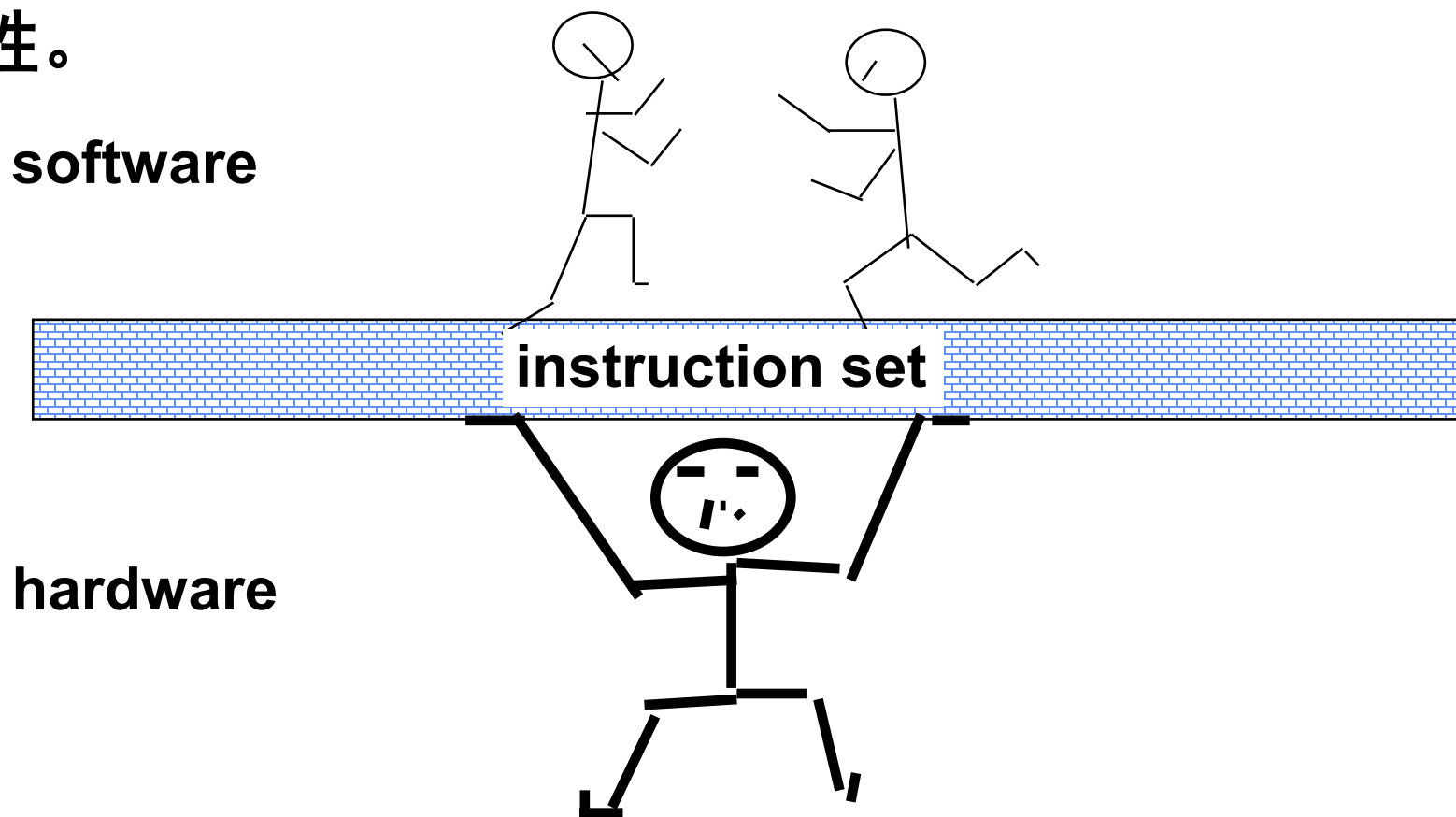
- Instruction Set Architecture (ISA) 设计即体系结构设计需要关注并确定：
 - 寄存器组织、如何访问存储器、寻址方式、指令操作数、硬件支持的操作种类、指令编码方式

◆ 目前的观点：

- 根据目标机器的特定需求，在成本、功耗、可用性等约束下最大化机器性能
- 包括ISA，计算机组织（微体系结构），硬件实现

ISA: a Critical Interface

- 计算机系统结构：计算机系统的软、硬件的界面
- 即机器语言程序员所看到的传统机器级所具有的属性。

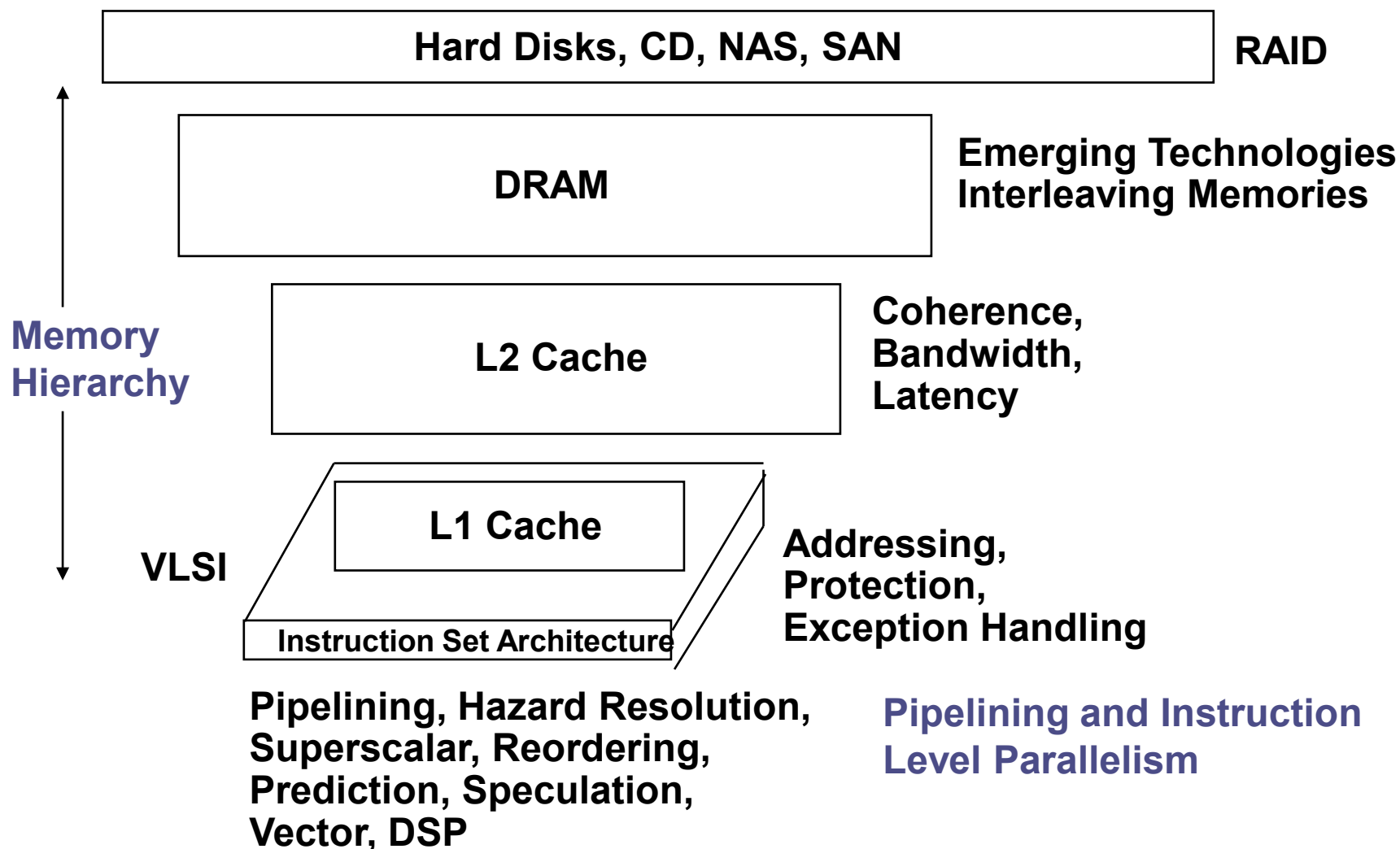


1.1.3 计算机组成和计算机实现

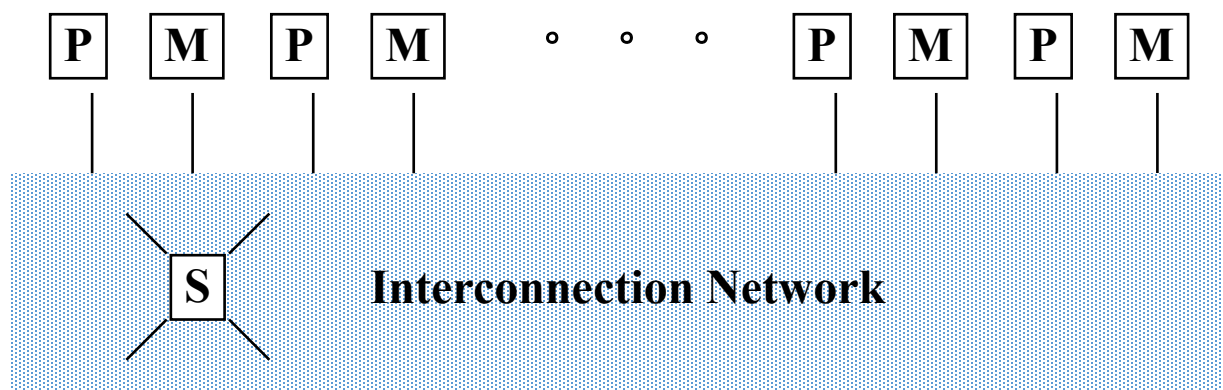
- ◆ **计算机组成** (Organization): ISA的逻辑实现
 - 物理机器级中的数据流和控制流的组成以及逻辑设计等
- ◆ **计算机实现** (Implementation): 计算机组成的物理实现
 - CPU, MEMORY等的物理结构, 器件的集成度、速度, 模块、插件、底板的划分与连接、信号传输、电源、冷却及整机装配技术等
- ◆ 例如
 - 确定指令系统中是否有乘法指令 (**Architecture**)
 - 确定用加法器实现乘法, 还是用专门的乘法实现 (**Organization**)
 - 器件的选定及所用的微组装技术 (**Implementation**)

Research Topics

Input/Output and Storage



Research Topics



Processor-Memory-Switch

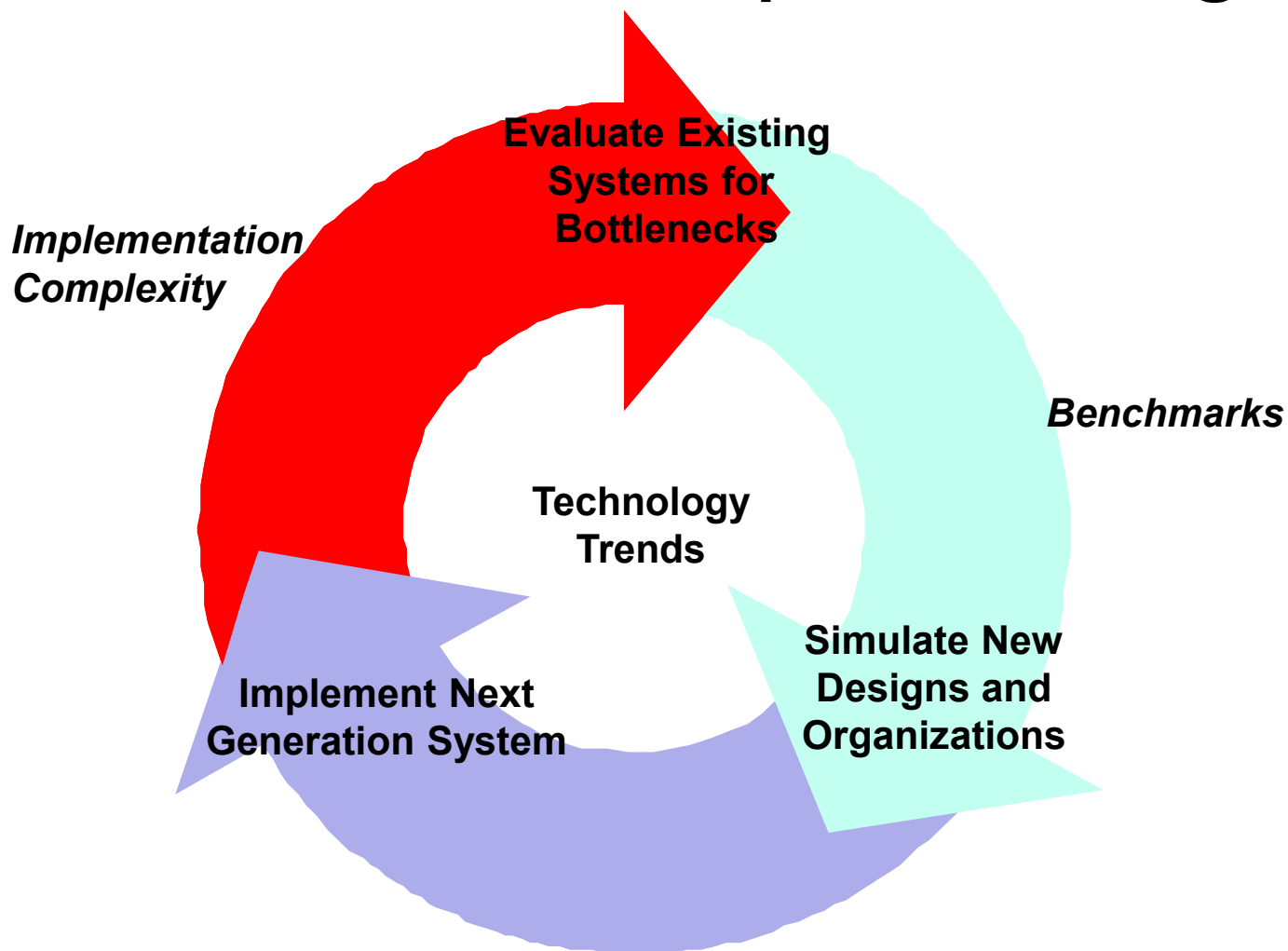
Multiprocessors
Networks and Interconnections

**Shared Memory,
Message Passing,
Data Parallelism**

Network Interfaces

**Topologies,
Routing,
Bandwidth,
Latency,
Reliability**

The Tasks of a Computer Designer



Basic principle: balance Cost /Performance

1.1.4 计算机系统的分类

◆ 常见的计算机系统结构分类法：

- 冯氏分类法、Flynn分类法、Handler分类法

◆ 冯氏分类法

- 用系统的最大并行度对计算机进行分类。
- **最大并行度 P_m** ：计算机系统在单位时间内能够处理的最大的二进制位数。
 - 用平面直角坐标系中的一个点代表一个计算机系统，
 - 其横坐标表示字宽（n位），
 - 纵坐标表示一次能同时处理的字数（m字）。
 - $m \times n$ 就表示了其最大并行度。



冯氏分类法

冯氏分类法

◆ 4类不同最大并行度的计算机系统结构

- 字串位串： $n=1$, $m=1$ 。(第一代纯串行计算机)
- 字串位并： $n>1$, $m=1$ 。这是传统的单处理机，同时处理单个字的多个位，如16位、32位等。
- 字并位串： $n=1$, $m>1$ 。同时处理多个字的同一位（位片）
- 字并位并： $n>1$, $m>1$ 。同时处理多个字的多个位

◆ 平均并行度

- 取决于系统的运用程度，与应用程序有关。
- 假设每个时钟周期内能同时处理的二进制位数为 P_i ，则 T 个时钟周期内的平均并行度为：

$$P_a = \frac{\sum_{i=1}^T P_i}{T}$$

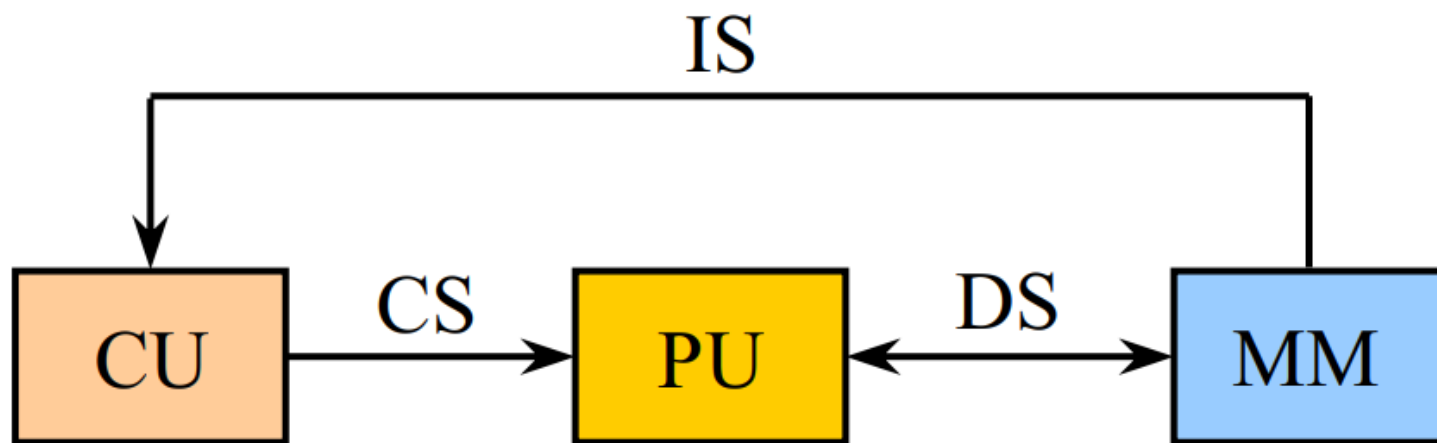
- 平均利用率定义为： $\mu = \frac{P_a}{P_m} = \frac{\sum_{i=1}^T P_i}{TP_m}$

Flynn分类法

- ◆ 按照指令流和数据流的多倍性进行分类。
 - 指令流：计算机执行的指令序列。
 - 数据流：由指令流调用的数据序列。
 - 多倍性：在系统受限的部件上，同时处于同一执行阶段的指令或数据的最大数目。
- ◆ Flynn分类法把计算机系统的结构分为4类：
 - 单指令流单数据流(SISD)
 - 单指令流多数据流(SIMD)
 - 多指令流单数据流(MISD)
 - 多指令流多数据流(MIMD)

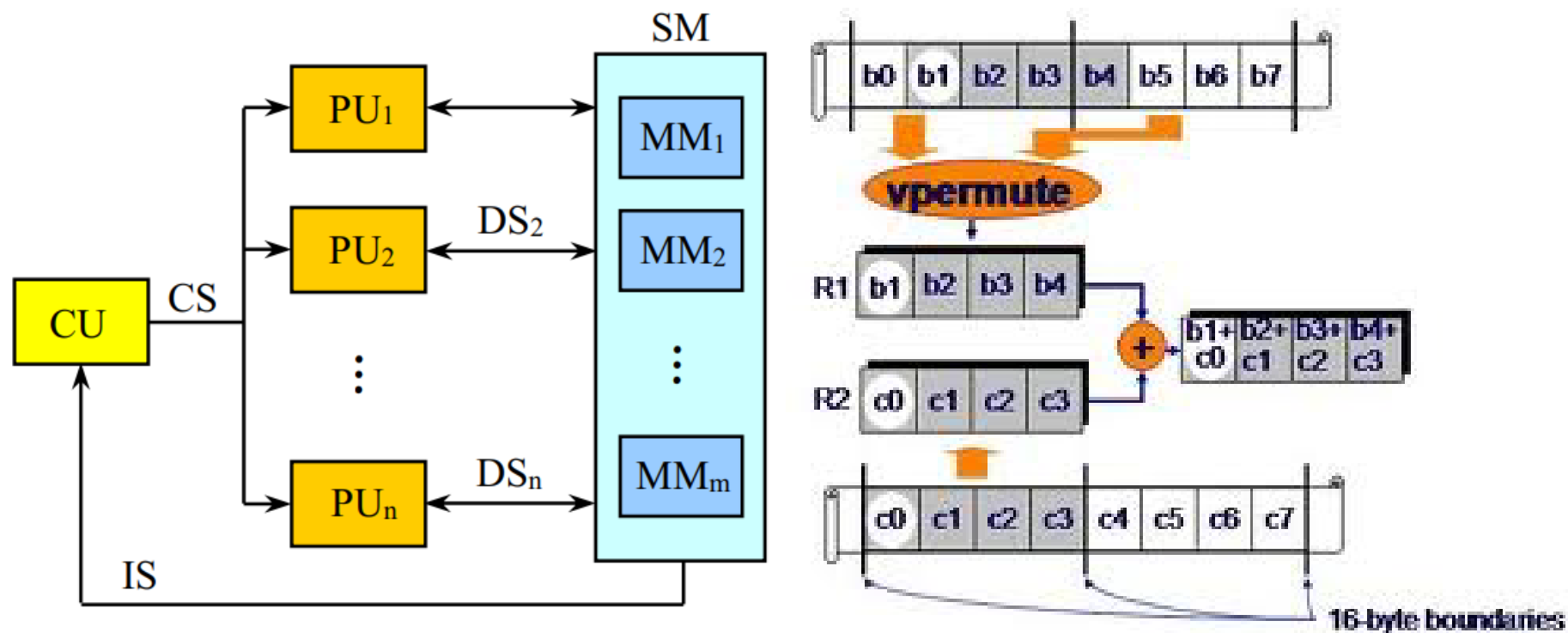
SISD

IS: 指令流
DS: 数据流
CS: 控制流
CU: 控制部件
PU: 处理部件
MM和SM: 存储器



Ex: PDP-11、IBM-360/370、PC 8086、Z-80

SIMD

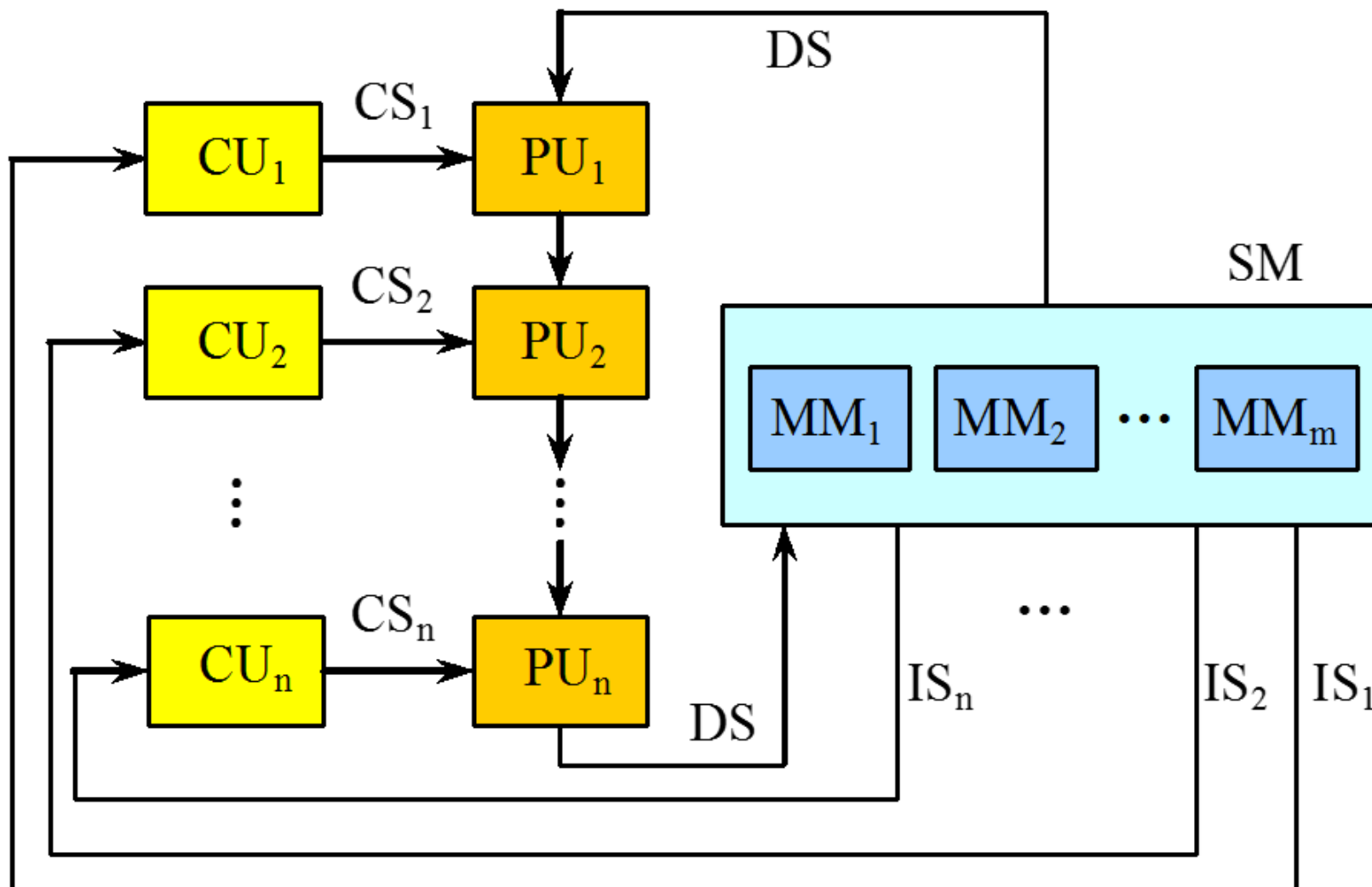


◆ Ex: Vectors Operate (+, -, *)

ILLIAC IV(64 Units) Array Machine

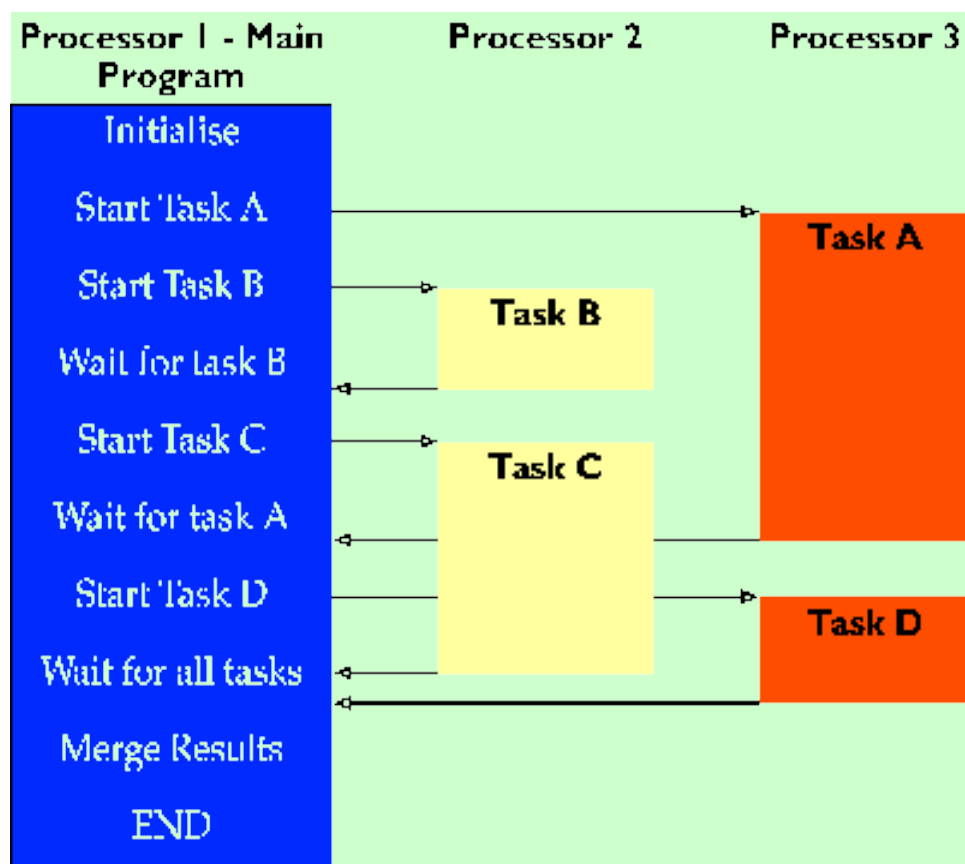
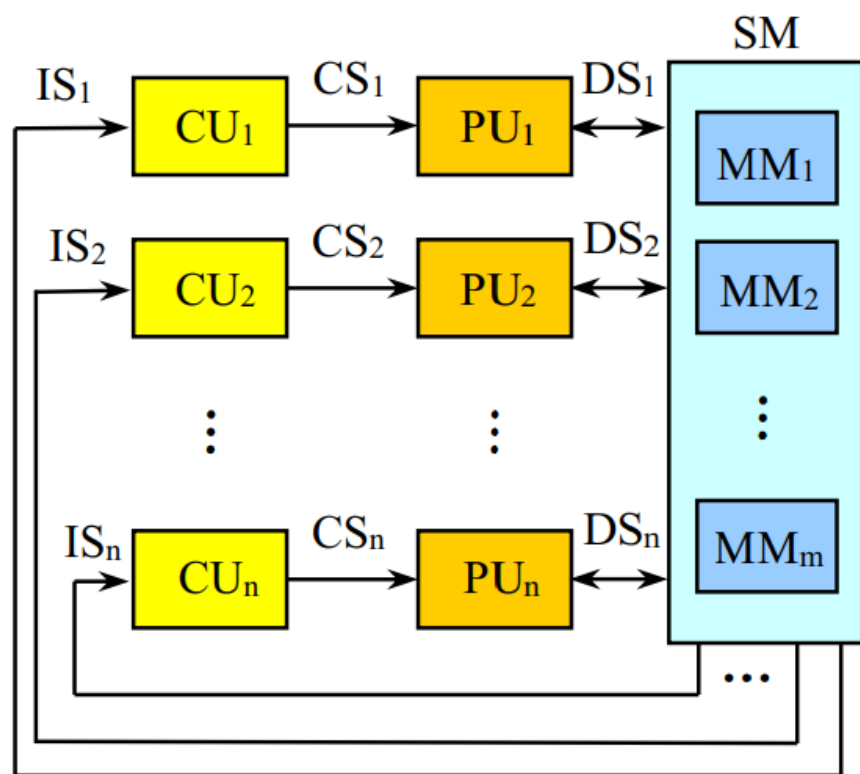
Pentium's (V)P[ADD/SUB] [B/W/D/Q]

MISD



人为划分，目前没有实际的计算机

MIMD



EX: IBM 3081/3084、Univac 1100/80, Cray-2 etc.

多处理机属于MIMD

1.2 计算机系统的设计

◆ 1.2.1 计算机系统设计的定量原理

- 以经常性事件为重点
 - Amdahl定律
 - CPU性能公式
 - 程序的局部性原理
- ### ◆ (1) 以经常性事件为重点
- 对经常发生的情况采用优化方法的原则进行选择，以得到更多的总体上的改进。
 - 优化是指分配更多的资源、达到更高的性能或者分配更多的电能等。

(2) Amdahl定律

◆ Amdahl定律

- 加快某部件执行速度所能获得的系统性能加速比，受限于该部件的执行时间占系统中总执行时间的百分比。
- 可用于计算当对计算机系统中的某个部分进行改进后，能获得多少总体性能的提升。

◆ 系统性能加速比：

$$\text{加速比} = \frac{\text{系统性能改进后}}{\text{系统性能改进前}} = \frac{\text{总执行时间改进前}}{\text{总执行时间改进后}}$$

(2) Amdahl定律

◆ 加速比依赖于两个因素

- **可改进比例**：在改进前的系统中，可改进部分的执行时间在总的执行时间中所占的比例。
 - 它总是小于等于1
 - 例如：一个需运行60秒的程序中有20秒的运算可以加速，那么这个比例就是 $20/60$ 。
- **部件加速比**：可改进部分改进以后性能提高的倍数。它是改进前所需的执行时间与改进后执行时间的比。
 - 一般情况下部件加速比是大于1的
 - 例如：若系统改进后，可改进部分的执行时间减少为5秒，则该部件加速比为 $20/5$ 。

(2) Amdahl定律

◆ 改进后程序的总执行时间

总执行时间_{改进后} = 不可改进部分的执行时间

+ 可改进部分改进后的执行时间

= (1 - 可改进比例) × 总执行时间_{改进前}

+ $\frac{\text{可改进比例} \times \text{总执行时间}_{\text{改进前}}}{\text{部件加速比}}$

= $\left[(1 - \text{可改进比例}) + \frac{\text{可改进比例}}{\text{部件加速比}} \right] \times \text{总执行时间}_{\text{改进前}}$

(2) Amdahl定律

◆ 加速比

- 系统加速比为改进前与改进后总执行时间之比

$$\begin{aligned}\text{加速比} &= \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}} \\ &= \frac{1}{(1 - \text{可改进比例}) + \frac{\text{可改进比例}}{\text{部件加速比}}}\end{aligned}$$

(2) Amdahl定律-应用1

- ◆ 例1.1 将计算机系统中某一功能的处理速度提高到原来的20倍，但该功能的处理时间仅占整个系统运行时间的40%，则采用此提高性能的方法后，能使整个系统的性能提高多少？

解 由题可知，可改进比例 = 40% = 0.4，

部件加速比 = 20

根据Amdahl定律可知：

$$\text{总加速比} = \frac{1}{(1-0.4) + \frac{0.4}{20}} = 1.613$$

采用此提高性能的方法后，能使整个系统的性能提高到原来的1.613倍。

(2) Amdahl定律-应用2

- ◆ 例1.2 某计算机系统采用浮点运算部件后，使浮点运算速度提高到原来的20倍，而系统运行某一程序的整体性能提高到原来的5倍，试计算该程序中浮点操作所占的比例。

解 由题可知，部件加速比 = 20，系统加速比 = 5

根据Amdahl定律可知

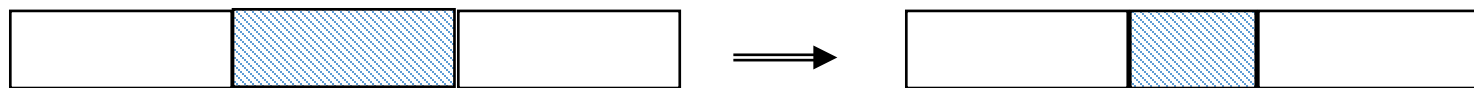
$$\frac{1}{(1 - \text{可改进比例}) + \frac{\text{可改进比例}}{20}} = 5$$

由此可得：可改进比例 = 84.2%

即程序中浮点操作所占的比例为84.2%。

(2) Amdahl定律推论

- ◆ Amdahl定律：一种性能改进的递减规则
 - 如果仅仅对计算任务中的一部分做性能改进，则改进得越多，所得到的总体性能的提升就越有限



- ◆ **重要推论：**

- 如果只针对整个任务的一部分进行改进和优化，那么所获得的加速比不超过

$$\frac{1}{1 - \text{可改进比例}}$$

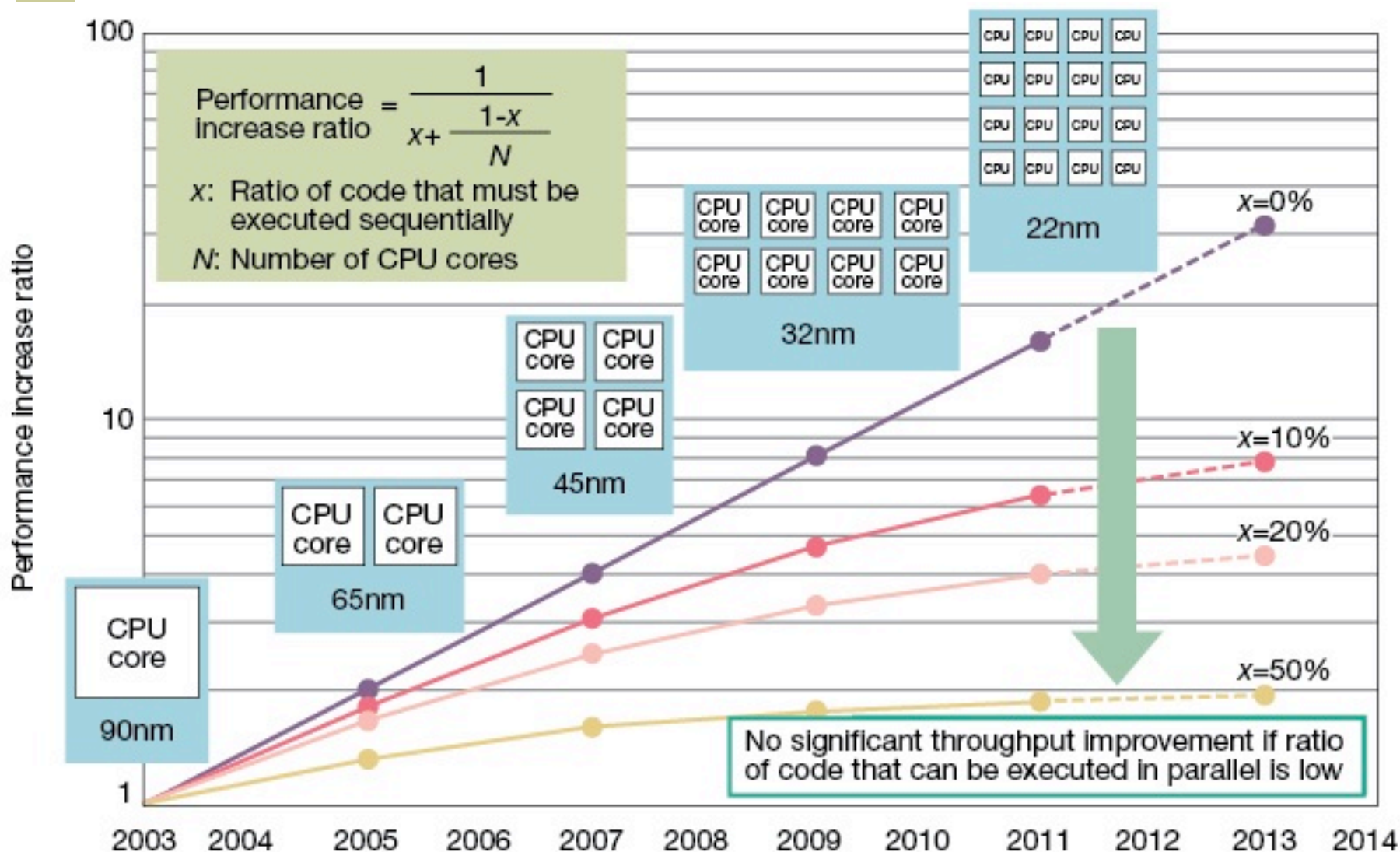


Fig 3 Amdahl's Law an Obstacle to Improved Performance Performance will not rise in

(3) CPU性能公式

- ◆ 执行一个程序所需的CPU时间
 - CPU时间 = 执行程序所需的时钟周期数 $\times$ 时钟周期时间
 - 其中，时钟周期时间是系统时钟频率的倒数。
- ◆ 每条指令执行的平均时钟周期数 CPI (Cycles Per Instruction)
 - CPI = 执行程序所需的时钟周期数 / IC
 - IC : 所执行的指令条数
- ◆ 程序执行的CPU时间可以写成
 - CPU时间 = $IC \times CPI \times$ 时钟周期时间

(3) CPU性能公式

- ◆ CPU的性能取决于3个参数
 - **时钟周期时间**：取决于硬件实现技术和计算机组成。
 - **CPI**：取决于计算机组成和指令集结构。
 - **IC**：取决于指令集结构和编译技术。

- ◆ 对CPU性能公式进行进一步细化

- 假设：计算机系统有 n 种指令

$$\text{CPU时钟周期数} = \sum_{i=1}^n (CPI_i \times IC_i)$$

- 其中， CPI_i ：第 i 种指令的处理时间；
 IC_i ：在程序中第 i 种指令出现的次数

(3) CPU性能公式

CPU时间 = 执行程序所需的时钟周期数 $\times$ 时钟周期时间

$$= \sum_{i=1}^n (CPI_i \times IC_i) \times \text{时钟周期时间}$$

◆ CPI 可以表示为

$$CPI = \frac{\text{时钟周期数}}{IC} = \frac{\sum_{i=1}^n (CPI_i \times IC_i)}{IC} = \sum_{i=1}^n \left(CPI_i \times \frac{IC_i}{IC} \right)$$

- 其中, (IC_i/IC) 反映了第*i*种指令在程序中所占的比例。

(3) CPU性能公式-举例

◆ 例1.4 考虑条件分支指令的两种不同设计方法：

(1) CPU_A：通过比较指令设置条件码，然后测试条件码进行分支。

(2) CPU_B：在分支指令中包括比较过程。

在这两种CPU中，条件分支指令都占用2个时钟周期，而所有其他指令占用1个时钟周期。

对于CPU_A，执行的指令中分支指令占20%；由于每条分支指令之前都需要有比较指令，因此比较指令也占20%。

由于CPU_A在分支时不需要比较，因此CPU_B的时钟周期时间是CPU_A的1.25倍。

问：哪一个CPU更快？

如果CPU_B的时钟周期时间只是CPU_A的1.1倍，哪一个CPU更快呢？

(3) CPU性能公式-举例

	指令数	时钟周期	CPI	总CPU时间
CPU _A	总指令数: IC_A 分支指令: $0.2IC_A$; 比较指令: $0.2IC_A$; 其它指令: $0.6IC_A$	CLK_A	$0.2*2+0.8*1$ $=1.2$	$IC_A*1.2*CLK_A$ $=1.2*IC_A*CLK_A$
CPU _B	分支指令: $0.2IC_A$; 其它指令: $0.6IC_A$; 总指令数 $IC_B=0.8IC_A$	$CLK_B=$ $1.25CLK_A$	$0.25*2$ $+0.75*1$ $=1.25$	$IC_B*1.25*CLK_B$ $=0.8IC_A*1.25*1.25CLK_A$ $=1.25*IC_A*CLK_A$
CPU _{B'}		$CLK_B=$ $1.1CLK_A$		$IC_B*1.25*CLK_B$ $=0.8IC_A*1.25*1.1CLK_A$ $=1.1*IC_A*CLK_A$

- ◆ 虽然CPU_B指令条数少，但CPU_A的时钟周期更短
 - 程序在CPU_A上运行的更快
- ◆ CPU_B时钟周期调整后，程序在CPU_{B'}上运行的更快

(4) 程序的局部性原理

- ◆ 程序执行时所访问的存储器地址分布不是随机的，而是相对地簇聚。
- ◆ 常用的一个经验规则
 - 程序执行时间的80%都是在执行程序中20%的代码
- ◆ 程序的时间局部性
 - 程序即将用到的信息很可能就是目前正在使用的信息。
- ◆ 程序的空间局部性
 - 程序即将用到的信息很可能与目前正在使用的信息在空间上相邻或者临近。

1.2.2 计算机系统设计者的主要任务

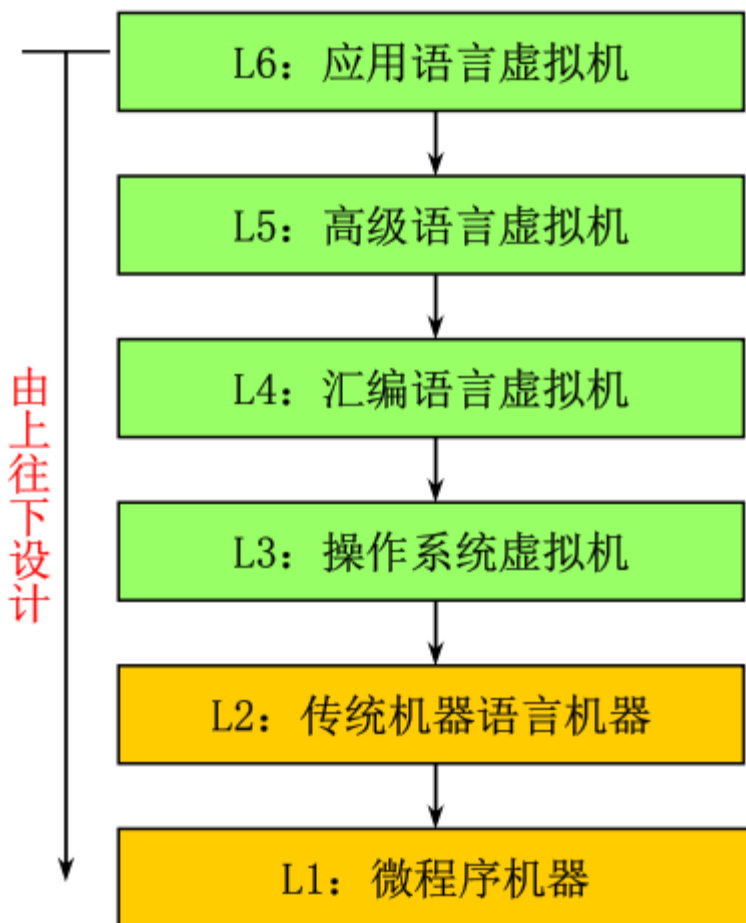
- ◆ 计算机系统设计者的任务包括：指令系统的设计、数据表示的设计、功能的组织、逻辑设计以及其物理实现等。
- ◆ 设计一个计算机系统大致要完成3个方面的工作
 - 确定用户对计算机系统的功能、价格和性能的要求
 - 设计出能满足用户的功能需求、有较长的生命周期、且又具有很高的性能价格比的系统。
 - 要考虑应用领域；软件兼容性；操作系统需求；采用哪些标准
 - 软硬件功能分配。软、硬件实现的优缺点，要进行折中
 - 设计出生命周期长的系统结构。要有一定的前瞻性

ISA需说明的主要内容

- ◆ 包含的主要内容：
 - Memory addressing
 - Addressing modes
 - Types and sizes of operands
 - Operations
 - Control flow instructions
 - Encoding an ISA
 -
- ◆ 优秀的ISA所具有的特征
 - 可持续用于很多代机器上 (portability)
 - 可以适用于多个领域 (generality)
 - 对上层提供方便的功能 (convenient)
 - 可以由下层有效地实现 (efficient)

1.2.3 计算机系统设计的主要方法

◆ (1) “由上往下” (top-down) 设计



第一步：确定这一级的基本特征

第二步：设计或选择面向这种应用的高级语言

第三步：设计适合所用高级语言编译的中间语言

第四步：设计面向这种应用的操作系统

第五步：设计面向所用编译程序和操作系统的机器语言

第六步：设计面向机器语言的微指令及其硬件实现

- 适合于**专用机**的设计，而不适合通用机的设计。

1.2.3 计算机系统设计的主要方法

- ◆ (2) “由下往上” (bottom-up) 设计
 - 软件技术完全处于被动状态，这会造成软件和硬件的脱节，使整个系统的效率降低。
 - 在早期被采用得比较多，现在已经很少被采用了
- ◆ (3) “从中间开始” (middle-out) 设计
 - “由上往下”和“由下往上”设计方法的主要**缺点**：
软、硬件设计分离和脱节
 - **解决方法**：综合考虑软、硬件的分工，从中间开始设计。
“中间”：层次结构中的软硬件的交界面，目前一般是在传统机器语言机器级与操作系统机器级之间。
 - 首先要进行软、硬件功能分配，确定好这个界面。
 - 软件设计者开始往上设计；硬件设计者开始往下设计

1.3 计算机系统的性能评测

- ◆ X比Y性能高的含义是什么?
- ◆ Ferrari vs. School Bus?
- ◆ 2013 Ferrari 599 GTB
 - 2 passengers, 11.1 secs in quarter mile
- ◆ 2013 Type D school bus
 - 54 passengers, quarter mile time?
- ◆ 响应时间: e.g., time to travel $\frac{1}{4}$ mile
- ◆ 吞吐率/带宽: e.g., passenger-mi in 1 hour



性能的两层含义

Plane	DC to Paris	Speed	Passengers	Throughput (pmp)
Boeing 747	6.5 hours	610 mph	470	286,700
BAD/Sud Concorde	3 hours	1350 mph	132	178,200

- ◆ 哪个性能高？
- ◆ Time to do the task (Execution Time)
 - execution time, response time, **latency**
- ◆ Tasks per day, hour, week, sec, ns. .. (Performance)
 - **throughput**, bandwidth

这两者经常会有冲突的

(1) 执行时间和吞吐率

- ◆ 如何评测一台计算机的性能，与测试者看问题的角度有关。
 - 用户关心的是：单个程序的执行时间（执行单个程序所花的时间很少）
 - 数据处理中心的管理人员关心的是：吞吐率（在单位时间里能够完成任务很多）

以时间 (Time) 度量性能

● 响应时间 (Response Time)

- 从任务开始到任务完成所经历的时间
- 通常由最终用户观察和测量
- 也称**Wall-Clock Time or Elapsed Time**
- **Response Time = CPU Time + Waiting Time (I/O, scheduling, etc.)**

● CPU执行时间 (CPU Execution Time)

- 指执行程序 (指令序列) 所花费的时间
- 不包括等待I/O或系统调度的开销
- 可以用秒(msec, μ sec, ...), 或
- 可以用相对值 (CPU的时钟周期数 (clock cycles))

以吞吐率度量性能

- **吞吐率** = 单位时间完成的任务数
 - 任务数/小时、事务数/分钟、100Mbits/s
- 缩短任务执行时间可提高吞吐率 (**throughput**)
 - **Example:** 使用更快的处理器
 - 执行单个任务时间少 $\Rightarrow$ 单位时间所完成的任务数增加
- 硬件并行可提高吞吐率和响应时间(**response time**)
 - **Example:** 采用多处理器结构
 - 多个任务可并行执行, 单个任务的执行时间没有减少
 - 减少等待时间可缩短响应时间

性能

- ◆ 假设两台计算机为X和Y，X比Y快的意思是：
 - 对于给定任务，X的执行时间比Y的执行时间少。

X的性能是Y的 n 倍，即 $\frac{\text{执行时间}Y}{\text{执行时间}X} = n$

- ◆ 而**执行时间与性能成反比**，有 $\text{性能} = \frac{1}{\text{执行时间}}$

- ◆ 可得

$$n = \frac{\text{执行时间}Y}{\text{执行时间}X} = \frac{\frac{1}{\text{性能}Y}}{\frac{1}{\text{性能}X}} = \frac{\text{性能}X}{\text{性能}Y}$$

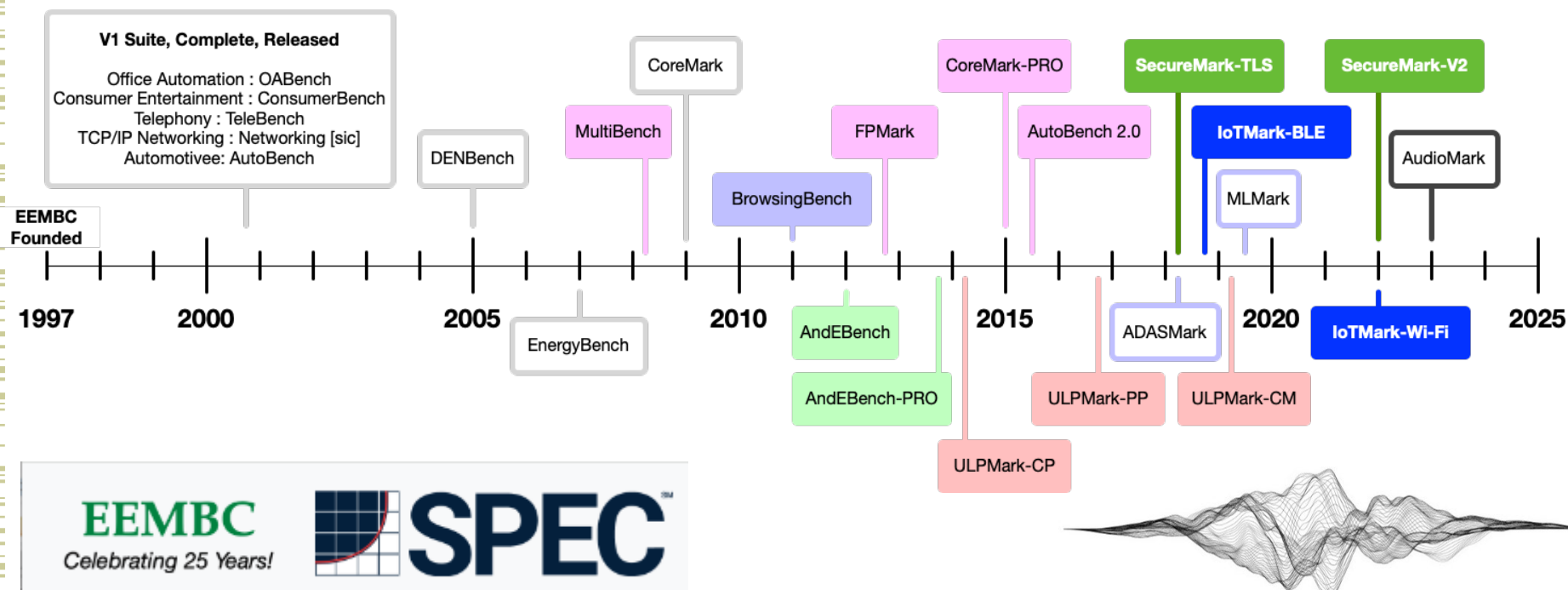
(2) 基准测试程序

- ◆ 五种类型的测试程序（**预测的精度逐级下降**）
 - 真实程序：这是最可靠的方法。
 - 修改过的程序：通过修改或改编真实程序来构造基准程序模块。原因：增强移植性或集中测试某种特定的系统性能
 - 核心程序(Kernels)：由从真实程序中提取的较短但很关键的代码构成。
 - Livermore Loops及LINPACK是其中使用比较广泛的例子
 - Lawrence Livermore National Laboratory. It consists of 24 do loops
 - 小测试程序(toy programs)：一般在100行以内。
 - 合成测试程序(Synthetic benchmarks)：首先对大量的应用程序中的操作进行统计，得到各种操作比例，再按这个比例人造出测试程序。
 - Whetstone与Dhrystone是最流行的合成测试程序

基准测试程序

◆ Embedded Microprocessor Benchmark Consortium (EEMBC)

- <https://www.eembc.org/products/>
- 共发布了46个性能测试向量，分别应用于电信、网络、消费性产品、办公室设备和汽车电子这五大领域。



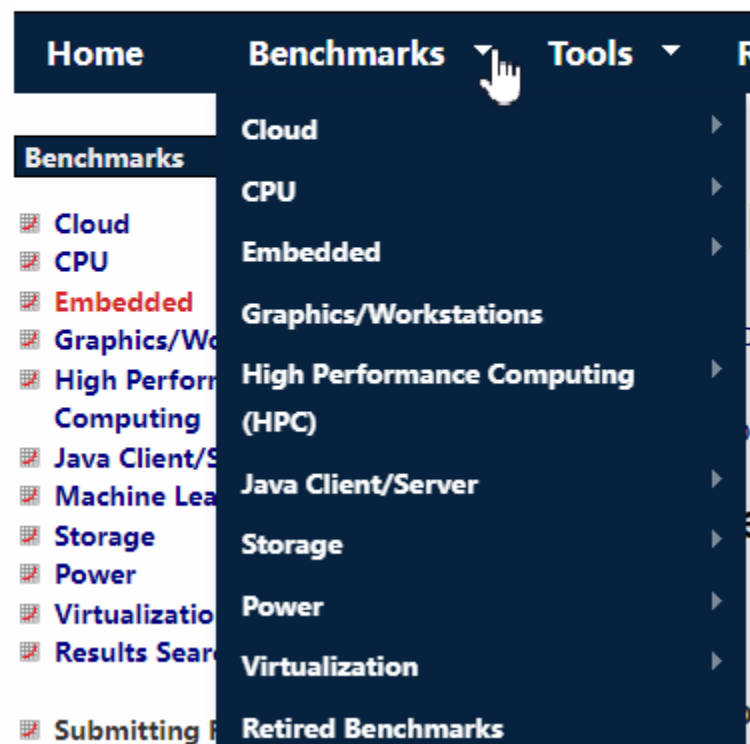
基准测试程序

- ◆ Standard Performance Evaluation Corporation (SPEC)

- <https://www.spec.org/benchmark>

- ◆ Desktop Benchmarks

- SPEC2017
 - SPEC2006
 - SPEC2000
 - SPEC 95
 - SPEC 92
 - SPEC 89





	Benchmark name by SPEC generation					
	SPEC2017	SPEC2006	SPEC2000	SPEC95	SPEC92	SPEC89
GNU C compiler	←					gcc
Perl interpreter	←			perl		espresso
Route planning	←		mcf			li
General data compression	XZ		bzip2		compress	eqntott
Discrete Event simulation - computer network	←	omnetpp	vortex	go	sc	
XML to HTML conversion via XSLT	←	xalancbmk	gzip	ijpeg		
Video compression	X264	h264ref	eon	m88ksim		
Artificial Intelligence: alpha-beta tree search (Chess)	deepsjeng	sjeng	twolf	integer programs		
Artificial Intelligence: Monte Carlo tree search (Go)	leela	gobmk	vortex			
Artificial Intelligence: recursive solution generator (Sudoku)	exchange2	astar	vpr			
		hmmer	crafty			
		libquantum	parser			
Explosion modeling	←	bwaves				fpppp
Physics: relativity	←	cactuBSSN				tomcatv
Molecular dynamics	←	namd				doduc
Ray tracing	←	povray				nasa7
Fluid dynamics	←	lbm				spice
Weather forecasting	←	wrf			swim	matrix300
Biomedical imaging: optical tomography with finite elements	parest	gamess		apsi	hydro2d	
3D rendering and animation	blender			mgrid	su2cor	
Atmosphere modeling	cam4	milc	wupwise	applu	wave5	
Image manipulation	imagick	zeusmp	apply	turb3d		
Molecular dynamics	nab	gromacs	galgel			
Computational Electromagnetics	fotonik3d	leslie3d	mesa			
Regional ocean modeling	roms	dealll	art			
		soplex	equake			
		calculix	facerec			
		GemsFDTD	ammp			
		tonto	lucas			
		sphinx3	fma3d			
			sixtrack			
						floating-point programs

SPEC Benchmarks

Category	Name	Measures performance of
Cloud	Cloud_IaaS 2016	Cloud using NoSQL database transaction and K-Means clustering using map/reduce
CPU	CPU2017	Compute-intensive integer and floating-point workloads
Graphics and workstation performance	SPECviewperf [®] 12	3D graphics in systems running OpenGL and Direct X
	SPECwpc V2.0	Workstations running professional apps under the Windows OS
	SPECapcSM for 3ds Max 2015™	3D graphics running the proprietary Autodesk 3ds Max 2015 app
	SPECapcSM for Maya [®] 2012	3D graphics running the proprietary Autodesk 3ds Max 2012 app
	SPECapcSM for PTC Creo 3.0	3D graphics running the proprietary PTC Creo 3.0 app
	SPECapcSM for Siemens NX 9.0 and 10.0	3D graphics running the proprietary Siemens NX 9.0 or 10.0 app
High performance computing	SPECapcSM for SolidWorks 2015	3D graphics of systems running the proprietary SolidWorks 2015 CAD/CAM app
	ACCEL	Accelerator and host CPU running parallel applications using OpenCL and OpenACC
	MPI2007	MPI-parallel, floating-point, compute-intensive programs running on clusters and SMPs
Java client/server	OMP2012	Parallel apps running OpenMP
	SPECjbb2015	Java servers
Power	SPECpower_ssj2008	Power of volume server class computers running SPECjbb2015
Solution File Server (SFS)	SFS2014	File server throughput and response time
	SPECsfs2008	File servers utilizing the NFSv3 and CIFS protocols
Virtualization	SPECvirt_sc2013	Datacenter servers used in virtualized server consolidation

(3) 性能比较

- ◆ 两个程序在A、B、C三台计算机上的执行时间

	A机	B机	C机	$W(1)$	$W(2)$	$W(3)$
程序1	1.00	10.00	20.00	0.50	0.909	0.999
程序2	1000.00	10.00	20.00	0.50	0.091	0.001
加权算术 平均值 $A_m(1)$	500.50	10.00	20.00			
加权算术 平均值 $A_m(2)$	91.91	10.00	20.00			
加权算术 平均值 $A_m(3)$	2.00	10.00	20.00			

性能比较

◆ 总执行时间

- 计算机执行所有测试程序的总时间

◆ 平均执行时间

- 各测试程序执行时间的算术平均值
- T_i : 第*i*个测试程序的执行时间
- n : 测试程序组中程序的个数

$$S_m = \frac{1}{n} \sum_{i=1}^n T_i$$

◆ 加权执行时间

- 各测试程序执行时间的加权平均值
- w_i : 第*i*个测试程序在测试程序组中所占的比重

$$A_m = \sum_{i=1}^n w_i \cdot T_i$$

$$\sum_{i=1}^n w_i = 1$$

性能比较

◆ 调和平均值法

- 如果性能使用速度（如MFLOPS）表示的，可以使用调和平均值法进行性能比较

- R_i : 由 n 个程序组成的工作负荷中执行第 i 个程序的速度; $R_i = 1/T_i$

- T_i : 第 i 个程序的执行时间

$$H_m = \frac{n}{\sum_{i=1}^n \frac{1}{R_i}} = \frac{n}{\sum_{i=1}^n T_i}$$

◆ 加权调和平均值公式

- W_i : 第 i 个程序在测试程序组中所占的比重

$$H_m = \left(\sum_{i=1}^n \frac{W_i}{R_i} \right)^{-1} = \left(\sum_{i=1}^n W_i T_i \right)^{-1}$$

性能的综合评价

- ◆ 规格化执行时间，采用几何平均

$$\sqrt[n]{\prod_{i=1}^n Execution_time_ratio_i}$$

- SPEC采用这种方法(SPECRatio)

$$SPEC\ Ratio = \frac{\text{Time on Reference Computer}}{\text{Time on Computer Being Rated}}$$

$$\frac{SPEC\ Ratio_A}{SPEC\ Ratio_B} = \frac{\frac{ExecutionTime_{Ref}}{ExecutionTime_A}}{\frac{ExecutionTime_{Ref}}{ExecutionTime_B}} = \frac{ExecutionTime_B}{ExecutionTime_A} = \frac{Performance_A}{Performance_B}$$

$$\text{Geometric Mean of SPEC Ratios} = \sqrt[n]{\prod_{i=1}^n SPEC\ Ratio_i}$$

性能的综合评价

$$\begin{aligned}\frac{\text{Geometric mean}_A}{\text{Geometric mean}_B} &= \frac{\sqrt[n]{\prod_{i=1}^n \text{SPECRatio } A_i}}{\sqrt[n]{\prod_{i=1}^n \text{SPECRatio } B_i}} = \sqrt[n]{\prod_{i=1}^n \frac{\text{SPECRatio } A_i}{\text{SPECRatio } B_i}} \\ &= \sqrt[n]{\prod_{i=1}^n \frac{\frac{\text{Execution time}_{\text{reference}_i}}{\text{Execution time}_{A_i}}}{\frac{\text{Execution time}_{\text{reference}_i}}{\text{Execution time}_{B_i}}}} = \sqrt[n]{\prod_{i=1}^n \frac{\text{Execution time}_{B_i}}{\text{Execution time}_{A_i}}} = \sqrt[n]{\prod_{i=1}^n \frac{\text{Performance}_{A_i}}{\text{Performance}_{B_i}}}\end{aligned}$$

- ◆ 几何平均的比率等于比率的几何平均
- ◆ 几何平均的比率 等于 性能比率的几何平均
 - 与参考机器的选择无关

SPECfp2000 Execution Times & SPEC Ratios

Benchmark	Ultra 5 Time (sec)	Opteron Time (sec)	SpecRatio Opteron	Itanium2 Time (sec)	SpecRatio Itanium2	Opteron/ Itanium2 Times	Itanium2/ Opteron SpecRatios
wupwise	1600	51.5	31.06	56.1	28.53	0.92	0.92
swim	3100	125.0	24.73	70.7	43.85	1.77	1.77
mgrid	1800	98.0	18.37	65.8	27.36	1.49	1.49
applu	2100	94.0	22.34	50.9	41.25	1.85	1.85
mesa	1400	64.6	21.69	108.0	12.99	0.60	0.60
galgel	2900	86.4	33.57	40.0	72.47	2.16	2.16
art	2600	92.4	28.13	21.0	123.67	4.40	4.40
equake	1300	72.6	17.92	36.3	35.78	2.00	2.00
facerec	1900	73.6	25.80	86.9	21.86	0.85	0.85
ammp	2200	136.0	16.14	132.0	16.63	1.03	1.03
lucas	2000	88.8	22.52	107.0	18.76	0.83	0.83
fma3d	2100	120.0	17.48	131.0	16.09	0.92	0.92
sixtrack	1100	123.0	8.95	68.8	15.99	1.79	1.79
apsi	2600	150.0	17.36	231.0	11.27	0.65	0.65
Geometric Mean			20.86		27.12	1.30	1.30

Geometric mean of ratios = 1.30 = Ratio of Geometric means = 27.12 / 20.86

为什么对规格化数采用几何平均?

	Computer A	Computer B	Computer C
Program P1 (secs)	1	10	20
Program P2 (secs)	1000	100	20
Total time (secs)	1001	110	40

	Normalized to A			Normalized to B			Normalized to C		
	A	B	C	A	B	C	A	B	C
Program P1	1.0	10.0	20.0	0.1	1.0	2.0	0.05	0.5	1.0
Program P2	1.0	0.1	0.02	10.0	1.0	0.2	50.0	5.0	1.0
Arithmetic mean	1.0	5.05	10.01	5.05	1.0	1.1	25.03	2.75	1.0
Geometric mean	1.0	1.0	0.63	1.0	1.0	0.63	1.58	1.58	1.0
Total time	1.0	0.11	0.04	9.1	1.0	0.36	25.03	2.75	1.0

1.4 计算机系统结构的发展

◆ 1.4.1 冯·诺依曼结构及其改进

- 存储程序原理的基本点：指令驱动
- 程序预先存放在计算机存储器中，机器一旦启动，就能按照程序指定的逻辑顺序执行这些程序，自动完成由程序所描述的处理工作。

1.4 计算机系统结构的发展

◆ 冯·诺依曼结构的主要特点

- 计算机以运算器为中心。
- 在存储器中，指令和数据同等对待。
- 存储器是按地址访问、按顺序线性编址的一维结构，每个单元的位数是固定的。
- 指令的执行是顺序的。
 - 一般是按照指令在存储器中存放的顺序执行。
 - 程序的分支由转移指令实现。
 - 由指令计数器PC指明当前正在执行的指令在存储器中的地址。
- 指令由操作码和地址码组成。
- 指令和数据均以二进制编码表示，采用二进制运算。

1.4 计算机系统结构的发展

◆ 对系统结构进行的改进

● 输入/输出方式的改进

输入输出方式

程序控制

程序等待

程序中断

DMA

成组传递

周期挪用

I/O 处理机

通道

外围处理机

● 采用并行处理技术

- 如何挖掘传统机器中的并行性？
- 在不同的级别采用并行技术。
- 例如：微操作级、指令级、线程级、进程级、任务级等。

● 存储器组织结构的发展

- 相联存储器与相联处理机
- 通用寄存器组
- 高速缓冲存储器Cache

● 指令系统的2个发展方向：

- 复杂指令集计算机CISC
- 精减指令集计算机RISC

1.4.2 软件对系统结构的影响

◆ 软件的可移植性：

- 一个软件可以不经修改或者只需少量修改就可以由一台机器移植到另一台机器上正确地运行。
- 差别只是执行时间的不同。
- 我们称这两台机器是软件兼容的。

◆ 实现可移植性的常用方法

- 采用系列机，模拟与仿真，统一高级语言。

◆ (1) 使用统一高级语言

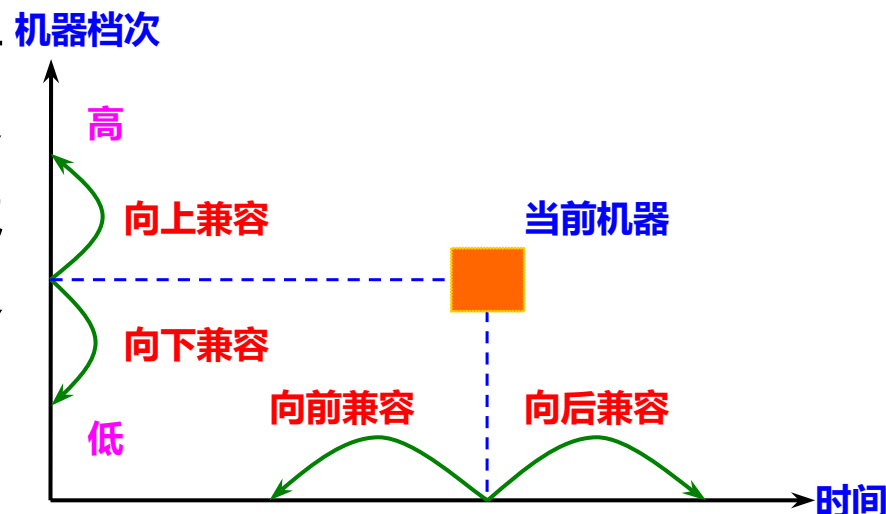
- 实现软件移植的一种理想的方法
- 较难实现

1.4.2 软件对系统结构的影响

◆ (2) 系列机

- 由同一厂家生产的具有相同的系统结构，但具有不同组成和实现的一系列不同型号的机器。
- 较好地解决软件开发要求系统结构相对稳定与器件、硬件技术迅速发展的矛盾。
- 向上（下）兼容：按某档机器编制的程序，不加修改就能运行于比它高（低）档的机器。
- 向前（后）兼容：按某个时期投入市场的某种型号机器编制的程序，不加修改地就能运行于在它之前（后）投入市场的机器。

向后兼容是系列机的根本特征



1.4.2 软件对系统结构的影响

- ◆ (3) 模拟和仿真
 - 使软件能在具有**不同系统结构**的机器之间相互移植。
 - 在一种系统结构上实现另一种系统结构。
 - 从指令集的角度来看，就是要在一种机器上实现另一种机器的指令集。
- ◆ 模拟：用软件的方法在一台现有的机器（称为宿主机）上实现另一台机器（称为虚拟机）的指令集。
 - 通常用解释的方法来实现。
 - 运行速度较慢，性能较差。
- ◆ 仿真：用一台现有机器（宿主机）上的微程序去解释实现另一台机器（目标机）的指令集。
 - 运行速度比模拟方法的快
 - 仿真只能在系统结构差距不大的机器之间使用

1.4.3 器件发展对系统结构的影响

- ◆ 推动计算机系统结构不断发展的最活跃因素
- ◆ 摩尔定律
 - 集成电路芯片上所集成的晶体管数目每隔18个月就翻一番。
- ◆ 计算机的分代主要以器件作为划分标准。
 - 它们在器件、系统结构和软件技术等方面都有各自的特征。
 - 例如Cache
 - P22, 表1.2给出了5代计算机的典型特征

1.4.4 应用对系统结构的影响

- ◆ 不同的应用对计算机系统结构的设计提出了不同的要求。
- ◆ 应用需求是促使计算机系统结构发展的最根本的动力。
- ◆ 一些特殊领域：需要高性能的系统结构
 - 高结构化的数值计算
 - 气象模型、流体动力学、有限元分析
 - 非结构化的数值计算
 - 蒙特卡洛模拟、稀疏矩阵
 - 实时多因素问题
 - 语音识别、图象处理、计算机视觉
 - 大存储容量和输入输出密集的问题
 - 数据库系统、事务处理系统
 - 图形学和设计问题
 - 计算机辅助设计
 - 人工智能
 - 面向知识的系统、推理系统